

Universidad Autónoma de Chiriquí

Facultad de Economía

Escuela De Ciencias de Computación



Lic. en Gestión de Tecnología de la Información

¨Programación V – PROG 309¨

Estudiante:

Arian Cervantes

Jairo Chávez

Débora Cisneros

Daniela Pittí

Prof. Alexis Flores

Laboratorio #4

Caso: Plataforma de pedidos de comida a domicilio

Fecha de Entrega

11/09/2026

II Semestre

1. Objetivo del laboratorio

Analizar el caso de estudio asignado —una plataforma de pedidos de comida a domicilio—, seleccionar el patrón arquitectónico (o la combinación de patrones) más adecuado entre los diez estudiados en el eje temático 1, justificar por escrito dicha decisión frente a las alternativas descartadas, y elaborar un diagrama de alto nivel que comunique con claridad la arquitectura propuesta para su posterior defensa oral ante el docente y el grupo.

2. Resumen del caso de estudio

Una empresa emergente desea lanzar una plataforma de pedidos de comida a domicilio donde los usuarios exploren restaurantes, arman su pedido y lo pagan desde una aplicación web y una aplicación móvil. Los restaurantes deben recibir los pedidos en tiempo real y actualizar su estado (recibido, en preparación, en camino, entregado), y un repartidor debe ser notificado automáticamente cuando el pedido esté listo para ser recogido.

El caso plantea cuatro requerimientos determinantes para la decisión arquitectónica:

- La lógica de cálculo de precios, comisiones y promociones debe reutilizarse entre la app web, la app móvil y un futuro panel administrativo.
- El sistema debe escalar para atender picos de pedidos en horas de almuerzo y cena sin degradar el servicio.
- Cada cambio de estado de un pedido debe notificar automáticamente a varios módulos (cliente, restaurante, repartidor) sin que estos se conozcan directamente entre sí.
- El equipo de desarrollo necesita poder cambiar el diseño visual de la aplicación sin modificar la lógica de negocio ni la base de datos.

3. Patrón(es) seleccionados y justificación

3.1 Clasificación de requerimientos por aspecto arquitectónico

Requerimiento	Aspecto arquitectónico	Patrón(es) candidato(s)
Reutilización de la lógica de precios/comisiones/promociones entre 3 clientes	Separación de responsabilidades y reutilización de lógica	Capas
Escalar ante picos de pedidos (almuerzo/cena) sin degradar el servicio	Acceso de múltiples clientes a un servicio centralizado escalable	Cliente-Servidor
Notificación de cambios de estado a módulos que no se conocen entre sí	Comunicación desacoplada entre componentes	Bus de eventos (y, alternativamente, Broker)
Cambiar el diseño visual sin tocar lógica de negocio ni base de datos	Independencia entre interfaz y lógica	MVC

3.2 Selección: combinación de patrones

El equipo seleccionó una combinación de cuatro patrones, cada uno resolviendo una parte específica del problema, en lugar de forzar un único patrón para todo el sistema:

- Cliente-Servidor como topología general: la app web, la app móvil y el futuro panel administrativo actúan como clientes que consumen un mismo servicio centralizado a través de una API, lo que permite escalar el servidor (réplicas, balanceo de carga) de forma independiente a los clientes durante los picos de almuerzo y cena.
- Capas dentro del servidor central: se separan la capa de presentación (API REST), la capa de lógica de negocio (cálculo de precios, comisiones y promociones) y la capa de datos (persistencia de pedidos, restaurantes y usuarios). Esto permite que la lógica de negocio se reutilice sin duplicarse entre los tres clientes, que es exactamente lo que exige el caso.
- Bus de eventos para la notificación de estados: cada cambio de estado del pedido se publica como un evento; el cliente, el restaurante y el repartidor se suscriben a los eventos que les interesan sin conocerse ni depender directamente unos de otros, cumpliendo el requerimiento de notificación desacoplada.
- MVC dentro de cada aplicación cliente: separa la interfaz (Vista) de los datos y del flujo de interacción (Modelo/Controlador), de modo que el equipo de diseño pueda rediseñar la app sin tocar la lógica de negocio ni la base de datos, que residen en el servidor.

3.3 Por qué esta combinación es superior a las alternativas

Frente a usar un único patrón, la combinación anterior cubre los cuatro requerimientos con el menor acoplamiento posible: Capas resuelve la reutilización de lógica, Cliente-Servidor resuelve el acceso centralizado y escalable, MVC resuelve la independencia de la interfaz, y Bus de eventos resuelve la notificación entre módulos que no deben conocerse. Ningún patrón individual del listado de diez cubre, por sí solo, los cuatro frentes a la vez; intentarlo forzaría al equipo a resolver con código ad-hoc lo que un patrón adicional ya resuelve de forma estándar y bien documentada.

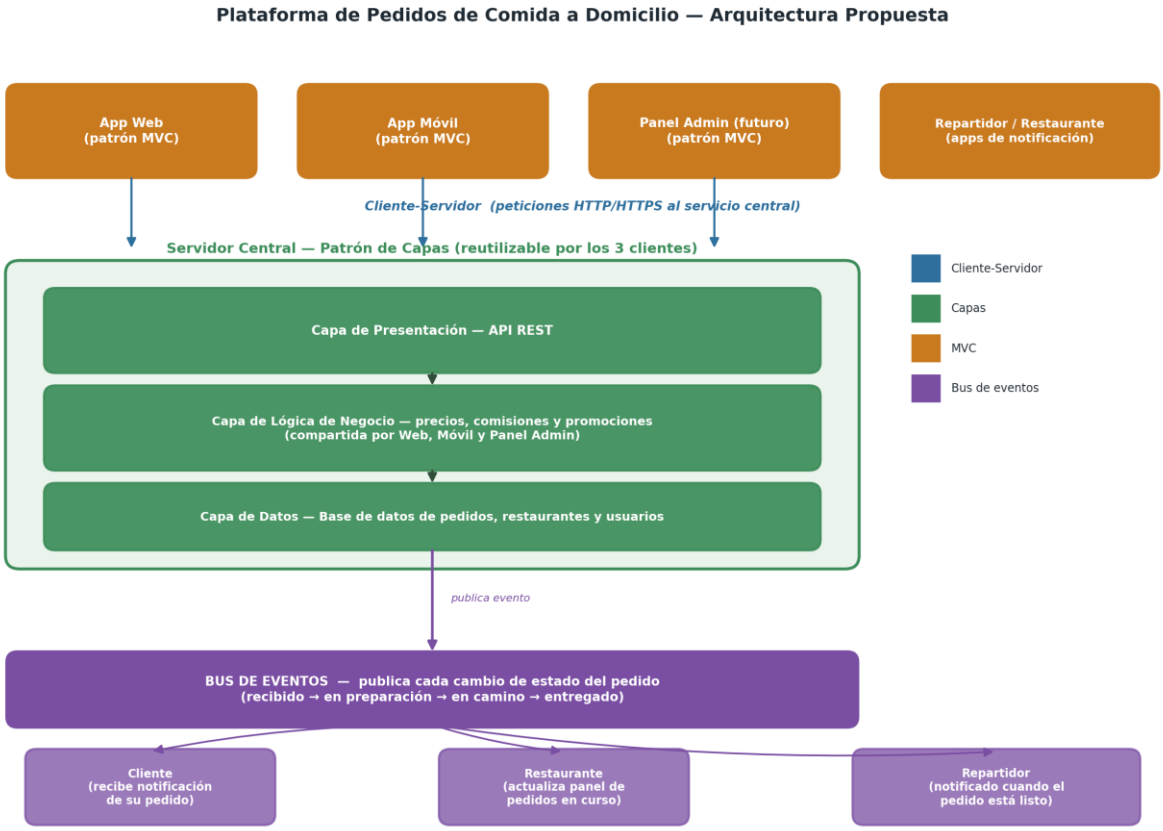
3.4 Patrones descartados y motivo

- Broker: también resuelve comunicación desacoplada entre componentes distribuidos, pero exige infraestructura de descubrimiento de servicios más compleja de la que este caso necesita; el Bus de eventos ofrece el mismo desacople con menor complejidad para un flujo de notificaciones relativamente simple (cuatro estados posibles).
- Maestro-esclavo: no aplica porque el caso no requiere dividir una tarea grande en subtareas idénticas ejecutadas en paralelo; procesar un pedido es un flujo secuencial de estados, no un cómputo paralelizable.
- Pipe-filter: se descarta porque el procesamiento del pedido no se compone de etapas de transformación de datos encadenadas; lo que varía es el estado de una misma entidad (el pedido), no datos que fluyen de un filtro a otro.
- Peer-to-peer: es inadecuado porque el negocio necesita control centralizado sobre pagos, inventario de restaurantes y consistencia de datos; un modelo sin punto único de control complicaría la seguridad de los pagos y la consistencia del estado de los pedidos.
- Blackboard: no hay múltiples estrategias heterogéneas de resolución compitiendo sobre un problema sin algoritmo claro; el proceso de pedido, cálculo de precios y notificación sigue reglas de negocio bien definidas, no una búsqueda exploratoria de solución.

- Intérprete: se descarta para el alcance actual porque no existe un lenguaje o notación propia que el sistema deba evaluar; no hay una gramática que interpretar en el flujo de pedidos.

4. Diagrama de arquitectura de alto nivel

El siguiente diagrama muestra los componentes principales de la arquitectura propuesta y cómo se comunican entre sí. Cada color identifica el patrón al que corresponde ese componente (ver leyenda dentro del propio diagrama): naranja para MVC (clientes), azul para Cliente-Servidor (comunicación cliente-servicio), verde para Capas (estructura interna del servidor) y morado para Bus de eventos (notificación de cambios de estado).



5. Respuestas a las preguntas de análisis

5.1 ¿Qué patrón(es) eligió su equipo y por qué es superior a las alternativas consideradas para este caso?

El equipo eligió la combinación Cliente-Servidor + Capas + MVC + Bus de eventos. Es superior porque cada requerimiento del caso encuentra respuesta en el patrón diseñado específicamente para ese tipo de problema (escalabilidad, reutilización de lógica, independencia de interfaz y notificación desacoplada), en vez de forzar un solo patrón a cubrir necesidades para las que no fue pensado, lo que evitaría duplicación de código o acoplamientos innecesarios entre módulos.

5.2 ¿Qué requerimiento del caso fue el más determinante para su decisión arquitectónica? Justifique.

El requerimiento más determinante fue la notificación de cada cambio de estado del pedido a varios módulos distintos (cliente, restaurante, repartidor) sin que estos se conozcan entre sí. Es el único requerimiento que exige explícitamente comunicación desacoplada entre múltiples receptores independientes, lo cual apunta de forma directa e inequívoca hacia un patrón de publicación-suscripción como el Bus de eventos; los demás requerimientos (reutilización de lógica, escalabilidad, independencia visual) admiten soluciones más convencionales dentro de Capas, Cliente-Servidor y MVC.

5.3 Si la aplicación tuviera que soportar, además, un asistente de voz que reciba pedidos, ¿qué patrón adicional consideraría y por qué?

Se consideraría el patrón Intérprete. Un asistente de voz necesita convertir una instrucción hablada en lenguaje natural (por ejemplo, "pide dos hamburguesas del restaurante X") en una estructura de datos que el sistema pueda procesar como un pedido formal; esto exige definir una gramática simple y relativamente estable (verbos de acción, nombres de producto, cantidades) y un componente que la evalúe línea por línea, que es exactamente el problema que resuelve el patrón Intérprete.

5.4 ¿Qué patrón de los diez estudiados sería claramente inadecuado para este caso? Explique por qué.

El patrón Peer-to-peer sería claramente inadecuado. Este caso exige control centralizado sobre aspectos críticos como el procesamiento de pagos, el inventario de restaurantes y la consistencia del estado de cada pedido; un modelo donde todos los nodos cooperan como iguales, sin un punto único de control, dificultaría garantizar la seguridad transaccional y la coherencia de los datos que una plataforma de comida a domicilio necesita.

5.5 ¿Cómo cambiaría su decisión si la empresa solo esperara operar en una sola ciudad con bajo volumen de pedidos?

Con bajo volumen y una sola ciudad, se simplificaría la arquitectura: se mantendría el patrón de Capas para conservar la separación de responsabilidades y la reutilización de lógica de negocio, pero el Bus de eventos podría sustituirse por llamadas directas (síncronas) entre módulos, ya que el beneficio de desacoplar productores y consumidores de eventos se aprovecha más en escenarios de mayor escala y variedad de suscriptores. El patrón MVC en los clientes se conservaría, pues la necesidad de cambiar el diseño visual sin tocar la lógica de negocio no depende del volumen de pedidos.

6. Conclusiones

- Ningún patrón arquitectónico aislado resuelve, por sí solo, todos los requerimientos de un sistema real; la pregunta relevante no es cuál es el patrón correcto, sino qué combinación responde mejor a los requerimientos específicos del caso.
- La combinación Cliente-Servidor + Capas + MVC + Bus de eventos permite atender simultáneamente la reutilización de lógica de negocio, la escalabilidad ante picos de demanda, la independencia de la interfaz visual y la notificación desacoplada entre módulos, que fueron los cuatro requerimientos clave del caso.
- Descartar explícitamente patrones como Peer-to-peer, Blackboard, Intérprete, Pipe-filter y Maestro-esclavo, y justificar por qué no encajan, fue tan importante como justificar los patrones elegidos, pues evidencia que la decisión fue deliberada y no la única alternativa considerada.
- Escenarios futuros (como incorporar un asistente de voz) o cambios en la escala del negocio (operar en una sola ciudad con bajo volumen) pueden requerir ajustar la combinación de patrones, lo que confirma que una arquitectura de software debe revisarse conforme evolucionan los requerimientos, no definirse como algo fijo desde el inicio.

7. Referencias

Huaman, W. C. (2018). Los 10 patrones comunes de arquitectura de software. Medium.

Blancarte, O. (2020). Introducción a la arquitectura de software.